UTN FRM · TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN · BASES DE DATOS II

Informe post-incidente

Simulacro de recuperación de Food Store — TP Unidad 6

Grupo 4: Belén Calvo · Hernán González · Elías Tello · Bruno Fiouchetta **Simulacros:** 28 al 30/10/2026

Entorno: PostgreSQL 17.11 · Windows 11 · Git Bash **Evidencias:** carpeta `mantenimiento/` del repo

RTO OBSERVADO	RPO OBSERVADO	TABLAS VALIDADAS	REGISTROS INVÁLIDOS
2,96 s	**66 s**	**5/5**	**0**
Previsto: 15 min	Previsto: 24 h (respaldo diario)	Conteos idénticos al respaldo	Productos y detalles

1 Respaldo utilizado

`foodstore_20261028_211058.dump` — generado el **28/10/2026 a las 21:10:58** por `backup_foodstore.sh` (formato custom `-Fc`, 9.428.332 bytes, 2 s). Junto al respaldo se guardó `foodstore_20261028_211058.conteo` con las filas por tabla en ese momento, que es la referencia de la validación. La base principal `foodstore` (100 MB) no se modificó: la restauración se hizo en `foodstore_restore_test`.

2 RPO y RTO: previstos y observados

INDICADOR	QUÉ MIDE	PREVISTO (LABORATORIO)	OBSERVADO
RPO	Cuántos datos se pueden perder: tiempo entre el último respaldo y el incidente	24 h (respaldo diario)	**66 s** — antigüedad del respaldo al iniciar el simulacro
RTO	Cuánto tarda la base en volver a estar disponible y validada	15 min	**2,96 s** — createdb + pg_restore 2,53 s + validaciones 0,43 s

El cronómetro arranca inmediatamente antes de `createdb foodstore_restore_test` y termina al pasar las validaciones. El RPO observado es bajo porque el simulacro se hizo un minuto después del respaldo; en operación real, con un respaldo diario, el peor caso sigue siendo 24 h de pérdida.

3 Resultado de las validaciones

TABLA	RESTAURADA	RESPALDO	
categoria	5	5	OK
cliente	20.005	20.005	OK
producto	50.011	50.011	OK
pedido	200.005	200.005	OK
detalle_pedido	499.263	499.263	OK

RESTRICCIÓN LÓGICA	RESULTADO
Productos inválidos (`precio < 0` o `stock < 0`)	0
Detalles inválidos (`cantidad <= 0` o `precio_unitario < 0`)	0

Los conteos coinciden además con el registro manual previo al primer respaldo (`conteo_antes_respaldo.txt`, 20:01).

Simulacro por integrante — cada uno restauró el mismo respaldo base en su PC (Hernán con PostgreSQL 18.6, el resto con 17.11)

INTEGRANTE	RESPALDO USADO	RTO OBSERVADO	RPO OBSERVADO	TABLAS · INVÁLIDOS	RESULTADO
Belén Calvo	`foodstore_20261028_211058`	2,96 s	66 s	5/5 coinciden · 0 / 0	OK
Hernán González	`foodstore_20261029_162309`	3,20 s	54 s	5/5 coinciden · 0 / 0	OK
Elías Tello	`foodstore_20261029_201505`	3,84 s	18 s	5/5 coinciden · 0 / 0	OK
Bruno Fiouchetta	`foodstore_20261030_130826`	4,29 s	15 s	5/5 coinciden · 0 / 0	OK

4 Problemas reales encontrados

Fin de línea de Windows (CRLF)

En el primer simulacro, `psql` devolvió cada fila terminada en `\r\n` y la tabla de validación se imprimió superpuesta (evidencia: `restore_test_intento1_crlf.txt`). Se corrigió quitando el `\r` con `tr -d '\r'`, y se agregó `.gitattributes` para que los `.sh` no se rompan al clonarlos en Windows.

Contadores de actividad en cero

`foodstore` se creó con `createdb -T bd2_tp3`, y esa copia no trae las estadísticas: `health_check.sql` mostró `n_live_tup = 0` en tablas con 499.263 filas. Esa observación justificó un `ANALYZE` (no `VACUUM FULL`), y después los contadores quedaron iguales a las filas reales.

5 Mejora para la próxima ejecución

Verificar cada respaldo en el momento de generarlo, con un tamaño mayor a 0 y `pg_restore -l` sobre el `.dump`. Así un archivo dañado se detecta el día del respaldo y no durante un incidente. Además, programar `backup_foodstore.sh` en el Programador de tareas de Windows con una retención de 7 días. Ya se aplicó una mejora: el simulacro valida contra el `.conteo` del respaldo y no contra la base viva, para evitar falsos fallos si `foodstore` recibe escrituras después del respaldo.

6 Trabajo asistido con IA y cómo se verificó

La primera versión de los scripts se armó con asistencia de Claude a partir de `SPEC_U6.md`. Cada comando se leyó y se probó antes de aceptarlo, incluido el camino de error del respaldo. **OpenCode** revisó los 4 scripts contra la spec (`revision_opencode.md`) y después implementó 3 cambios aceptados: un `ANALYZE` condicional que incluye `usuario`, un `.conteo` por respaldo y la validación contra ese `.conteo`. Se rechazó un hallazgo (`decisiones_revision.md`). **Verificación humana:** se corrigió el plan antes de aplicarlo (variables de `\echo` dentro de comillas, que `psql` no interpola). Se detectó que el diff que reportó OpenCode no coincidía con el archivo real (`rto_ms` en lugar de `rto_s`, comprobado con `grep`). Se ejecutó `maintenance.sql` dos veces: la primera decidió hacer `ANALYZE` y la segunda no hizo nada.

Entregables en `mantenimiento/`: `SPEC_U6.md` · `backup_foodstore.sh` · `health_check.sql` · `maintenance.sql` · `restore_test.sh` · `README.md` · evidencias y logs. Los `.dump` no se versionan por su tamaño; su creación y su restauración quedan registradas en `logs/` y `evidencias/`.